

Сборка кодинг-агента

День-0 база или жди сигнала — шесть развилок одного сквозного кейса

04

«Что в конфигурации агента стоит закладывать сразу, на день 0, как базовую практику — а что нельзя знать заранее, потому что оно специфично для вашего проекта?»

ПУСТАЯ ДИРЕКТОРИЯ → СЕМЬ ДОБАВЛЕНИЙ, ПОКА БЕЗ ИМЁН



РАЗДЕЛ 0 · ЗАДАЧА ОТ ЗАКАЗЧИКА

Задача приходит в формулировке заказчика: что должно получиться — есть, устройства кода, критериев готовности и границ действий — нет

«Нужен лендинг с формой заявки на демо-урок: одна статическая страница, поля 'имя' и 'email', кнопка отправки. Своего бэкенда нет — присылайте заявку на наш сервис приёма форм.»

«Отдаём эту формулировку агенту как есть — или сначала фиксируем требования?»

открытая дискуссия, не голосование

РАЗДЕЛ 0 · SPEC.MD

Требования раскладываются на функциональные и нефункциональные и ложатся в `spec.md` первым коммитом; `README` описывает устройство репозитория и заводится тем же коммитом

```
# spec.md – signup-landing
```

Функциональные требования

- Статическая страница-лендинг, форма заявки на демо-урок
- Поля: имя (обязательное), email (обязательное)
- Кнопка отправки; без бэкенда — заявка уходит на внешний сервис

Нефункциональные требования

- Критерий приёмки: форма реально доставляет заявку — проверено на проде, а не только «тест зелёный»
- Граница действий: не деплоить на прод без явного запроса

иллюстративный пример, не захваченный артефакт

«`README.md` заводится тем же коммитом, отдельно от `spec.md`. Спек — требования к продукту. `README` — устройство репозитория для того, кто в него зайдёт: человек или агент.»

РАЗДЕЛ 0 · СКВОЗНОЙ КЕЙС

Агент собрал работающий прототип по спеке без единого уточняющего вопроса — и структуру репозитория при этом выбрал сам, потому что `spec.md` её не описывает

```
$ ls -la signup-landing/
total 8
drwxr-xr-x 2 harness harness 4096 Sep 20 12:45 .
drwxr-xr-x 3 harness harness 4096 Sep 20 12:45 ..
```

реальный вывод, не макет

«собери лендинг с формой заявки на демо-урок: статическая HTML-страница, сборка Vite (`npm run build` → `dist/`), поля формы — имя и email, тест на Playwright, который проверяет, что форма не отправляется с пустыми обязательными полями (`tests/form.spec.ts`)» — по содержанию совпадает со `spec.md`

«Уточняющий вопрос не задавался. Все параметры задачи были однозначно выводимы из текста задачи, поэтому работа была начата сразу, без уточнений.» — строка из захваченной сессии

создано агентом

```
index.html
src/main.js
package.json
tests/form.spec.ts
```

из финального ответа

```
npm install
npm run build      # → dist/
npm run test:e2e   # playwright test
```

Настраивая агента в новом репозитории, каждый работает в одном из двух режимов — заложить базу сразу или дождаться реальной проблемы

Короткий срез по залу, прежде чем идти дальше.

«Когда вы настраиваете нового агента в новом репозитории — какой из двух режимов вам обычно ближе?»

А · «Закладываю базу сразу»

гейт готовности («что считается сделано»)

трекинг задач и решений

В · «Жду, пока не заболит»

обзор репозитория

вложенные файлы конвенций

сложная, многоуровневая память

разбор — на следующем слайде

РАЗДЕЛ 0 · РАЗБОР ДВУХ РЕЖИМОВ

Оба режима рабочие — вопрос в том, к какому типу решения они применяются: специфичное для проекта ждёт сигнала, универсальная гигиена заводится сразу

СПЕЦИФИЧНОЕ ДЛЯ ПРОЕКТА — ЖДАТЬ СИГНАЛА

обзор репозитория

вложенные файлы конвенций

сложная, многоуровневая память

УНИВЕРСАЛЬНАЯ ГИГИЕНА — ДЕНЬ 0

гейт готовности

трекинг задач и решений

«Лекция 3 говорила про тонкого агента — про отказ от структурной сложности: субагентов, скиллов, внешнего доступа. Там сигнал нужен. Универсальная гигиена — другая категория: здесь 'жди боли' работает против вас.»

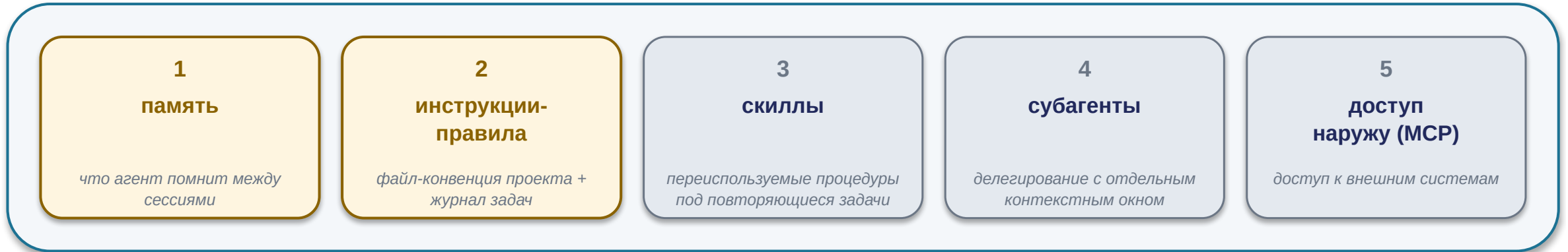
Кроме спеки и README в репозиторий в тот же день кладётся ещё три вещи — ни одна из них не ждала сигнала

Что заведено	Зачем именно сразу
спес.md + README.md	что должно получиться и как устроен репозиторий — первым коммитом, до задачи агенту
CLAUDE.md	файл инструкций агенту. В первый день в нём одна строка: критерий «готово» и требование проверить результат руками. Описания устройства репозитория в нём нет — для него ещё нет причины
DECISIONS.md	журнал решений проекта: что решили и почему. Чтобы отклонённый вариант не пришлось разбирать заново через месяц
Практика файла-на-задачу	на многошаговую задачу — отдельный файл с планом и отметками о ходе работы

Каждое из этого стоит сегодня одной строки. Цена отсутствия заранее неизвестна и обнаруживается задним числом: сигнала, по которому стало бы понятно, что пора, может не быть вовсе.

Дальше все четыре встречаются в работе поимённо — каждое в своей развилке.

Конфигурация агента раскладывается на пять слотов; сегодняшний проект задействует два — инструкции-правила и память



Сегодня в работе — два. Скиллы, субагенты, доступ наружу и механические слои (хук, процесс) — Семинар 5.

«Оставайтесь на самой нижней ступени, которая закрывает требование задачи. Поднимайтесь на следующую только тогда, когда можете назвать требование, которое текущая ступень не закрывает. Каждый подъём оплачивается новой стоимостью, новыми режимами отказа и новой поверхностью атаки.»

1

Раздел 1 · файл инструкций

- Кейс 1.1 · роль агента и репозиторий
- Кейс 1.2 · базовые процессы: гейты и цикл улучшения
- Кейс 1.3 · вложенные файлы по подпапкам

ОДИН ФАЙЛ, ТРИ РАЗВИЛКИ



Кейс 1.1 · роль агента и репозиторий

Чем заполняется пустой файл инструкций

ПЕРВОЕ РЕШЕНИЕ ПО КОНФИГУРАЦИИ



КЕЙС 1.1 · РОЛЬ И РЕПОЗИТОРИЙ

Прототип собран, следующая сессия будет завтра — в CLAUDE.md уже есть строка гейта готовности, остальное пусто, а стандартный первый раздел такого файла называется «Repository overview»

состояние репозитория

README.md

spec.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

CLAUDE.md ← гейт есть, остальное пусто

Прототип работает, задач по нему ещё на несколько дней. Разработчик заводит CLAUDE.md, чтобы агент в следующих сессиях работал с проектом одинаково, и открывает файл — там одна строка, гейт готовности, остальное пусто.

Первый раздел, с которого такой файл обычно начинается, — «Repository overview»: из чего собран сайт, где лежит форма, какая структура папок. Так устроено большинство файлов инструкций, которые можно открыть в публичных репозиториях.

Выбор из пяти вариантов: что из перечисленного попадёт в файл сейчас, перед следующей сессией

«Перед вами пустой CLAUDE.md репозитория signup-landing. Выберите из карточек, что попадёт в файл сейчас, перед следующей сессией агента.»

подробное описание
структуры репозитория

список технологий
и зависимостей

инструкцию, где лежит
форма и как её найти

неочевидную
конвенцию,
которую агент не
угадает сам

пока ничего — до
первого реального
сигнала

разбор — на следующем слайде

КЕЙС 1.1 · ИССЛЕДОВАНИЕ И РАЗБОР

Repository overview измеримо не повышает успешность и стоит контекста — а структурированное описание, если оно нужно, живёт в README.md и не дублируется в CLAUDE.md

Источник	Тезис	Сила
Gloaguen et al., arXiv:2602.11988	«are not helpful» — шагов до файла не падает; cost +20–23% без прироста success	Сильное: контролируемый эксперимент
Та же работа, абляция	Убрали README — LLM-overview стал полезен (+2,7%)	Подтверждает механизм
Lulla et al., arXiv:2601.20404	AGENTS.md про недискаверабельные конвенции: −28,6% runtime (124 PR)	Умеренно «за» — не общее описание
Shepard & Albrecht, arXiv:2606.20512	Статичное guidance (28,3%) хуже верифицированного (33,0%, $p<0,001$)	Косвенное

Вариант	Где работает	Почему рано — что вместо
«Описание структуры»	Почти нигде	Агент выведет сам за секунды; устареет
«Список технологий»	Версии критичны, не видны	Обычно тоже выводимо
«Инструкция, где форма»	Почти нигде на малом репо	Шесть файлов — вся структура на одном экране
«Неочевидная конвенция»	Когда реально есть	Такой конвенции ещё нет
«Пока ничего — до сигнала»	Здесь и сейчас	ЦЕЛЕВОЙ ОТВЕТ

README, не AGENTS.md: структурированное описание репозитория должно существовать — этот кейс не отменяет потребность в нём, он называет место. Оно живёт в README.md, заведённом первым коммитом, а не дублируется в CLAUDE.md. Отдельный «Repository overview» в файле инструкций не нужен: описание уже есть, просто в другом файле.

КЕЙС 1.1 · РЕШЕНИЕ

Описания структуры репозитория в CLAUDE.md не появляется: сигнала для него нет — а первое содержимое у файла появится уже в следующем кейсе, и по другой причине

состояние репозитория

README.md

spec.md

DECISIONS.md

index.html

package.json

src/main.js

tests/form.spec.ts

CLAUDE.md ← заведён, по-прежнему пуст

«Описания структуры репозитория в файле нет: сигнала для него нет. Пусто — не навсегда: часть содержимого CLAUDE.md появится в нём уже на этой неделе, и по другой причине — не по сигналу, а как базовая практика.»

Кейс 1.2 · базовые процессы

Гейт готовности и цикл «план → код → улучшение» — то, что лежит в файле с первого дня

ГЕЙТ ГОТОВНОСТИ В ФАЙЛЕ ИНСТРУКЦИЙ



Гейт готовности лежит в CLAUDE.md с дня 0 — и дважды подряд «готово» прозвучало до того, как доставку заявки проверили руками

День 0: гейт готовности записан в CLAUDE.md тем же днём, что и spec.md — «Всегда прогоняй тесты и вручную проверяй результат перед тем как сказать 'готово'».

Первая неделя: заказчик — «заявка не пришла». Агент читает код, правит обработчик, прогоняет тест — зелёный — «Готово». Тест проверяет только пустые поля. Разработчик напоминает про гейт — агент проверяет руками, чинит. Через два дня, другой баг — то же самое: тест зелёный, руками не проверено.

«Гейт-фраза лежала в файле все эти дни и загружалась в начало каждой сессии. Оба раза «готово» прозвучало до ручной проверки.»

Гейт-фраза уже есть с самого начала и не сработала — выбор из пяти вариантов, что делать дальше

«У вас гейт-фраза в CLAUDE.md есть с самого начала, а поведение агента всё равно расходится с ней второй раз подряд — заявка снова не проверена руками. Выберите, что вы сделаете дальше — из карточек ниже — так, чтобы это реально сработало в следующей сессии.»

перепишу фразу
капсом / жёстче

продублирую фразу
вторым пунктом

заведу отдельный
файл-чеклист рядом

передам проверку
механизму вне текста

смирюсь — сработает
не всегда, это норма

разбор — на следующем слайде

КЕЙС 1.2 · ИССЛЕДОВАНИЕ И РАЗБОР (1/2)

Текстовый гейт без тулинга не просто бесполезен, а вреден — регрессии выросли почти до 10%; ни один вариант, кроме передачи проверки механизму, не решает проблему системно

Источник	Тезис	Сила
SWE-Gate/SpecBench/CapCode/Verif. Horizon (4 препринта)	Согласие 4+ независимых исследований: агенты обходят видимые гейты	Сильное — консенсус
TDAD, arXiv:2603.17973	«Сначала пиши тесты» без тулинга — регрессии до 9,94%, хуже отсутствия	Сильное и контринтуитивное
Rethinking Agent-Generated Tests	Манипуляция промптом ради частоты тестов — без значимого эффекта	Умеренное — нулевой результат
Agent Scaffolding Beats Model Upgrades	Архитектура харнесса даёт +20 п.п.; смена модели — ~1 п.п.	Сильное — механизм против модели

Источник	Тезис	Сила
Self-planning, 2303.06689	Планирование перед кодом: +25,4% pass@1	Умеренно-сильное «за»
LLMs Cannot Self-Correct, ICLR'24	Самокоррекция без сигнала ухудшает	Сильное «против» self-review
arXiv:2604.10508	Self-repair с реальной проверкой: +4,9...+30 п.п.	Сильное «за» — с проверкой

Честный пробел: нет контролируемого эксперимента именно на изолированную фразу-гейт в CLAUDE.md — только best-practice наблюдения практиков.

Вариант	Где работает	Почему рано — что вместо
«Капсом / жёстче»	Нигде системно	TDAD: регрессии выросли до 9,94%
«Продублирую вторым пунктом»	Иногда чуть повышает шанс прочитать	Presence paradox: лишняя строка стоит контекста
«Отдельный файл-чеклист»	Помогает человеку, слабо — агенту	Тот же класс решения — снова текст, снова просьба
«Передам механизму вне текста»	Системно работает: +20 п.п.	Верное направление — механизм на следующем слайде
«Смирюсь, это нормально»	Приемлемо для некритичных предпочтений	Для правила, чьё нарушение стоит дорого — решение не решать

Правила в промпте — просьбы, правила в коде — законы; текстовый гейт ловит честную ошибку, но не держится под давлением

Правила в промпте — это просьбы, правила в коде — это законы. То, что мы написали, не стало законом от того, что мы записали его проактивно.

МОЖНО ЛИ СЕГОДНЯ ОБОЙТИСЬ БЕЗ ХУКА

Что гейт даёт	Ловит честную ошибку — «никто не вспомнил», не «агент обошёл проверку». Сработал 1 раз из 3: когда про него напомнили вслух
Чего не даёт	Не держится под давлением — срок, спешка. TDAD: усиление текста подняло регрессии до 9,94%; механизм даёт +20 п.п., модель — ~1 п.п.
Где граница	Хук нужен, когда цена нарушения высокая и предсказуемая — названы конкретное действие, команда и ущерб
Где мы в проекте	Действие названо: «готово» без проверки, дважды подряд, цена — сорванная доставка к показу. Текста хватает ровно до этого места

Зачем знать про хук, если сегодня его не ставим. Затем, чтобы не принять текстовый гейт за решение. Хук нужен не «для надёжности», а как ответ на названное действие с названной ценой: назвали — пора, не назвали — текста хватает.

Итоговый файл — тринадцать строк: критерий готово, гейт-предупреждение, честная оговорка предела; AGENTS.md — симлинк на тот же файл

CLAUDE.md

signup-landing: статический лендинг с формой заявки на демо-урок.
Готово = npm run build код 0, npx playwright test зелёный,
форма реально отправляет заявку на проде.

Safety / scope boundaries

- Никогда не запускать деплой на прод без явного запроса.

Build, test, verify

- To verify: открыть dist/index.html после сборки, вручную отправить форму с заполненными и с пустыми полями.

1

Файл грузится в начало каждой сессии целиком (официальный ориентир — до 200 строк, наш итог — 13)

2

@-импорты организуют файл, но не экономят контекст

3

AGENTS.md — симлинк на тот же файл (ln -s CLAUDE.md AGENTS.md). Острый край: cp -R без -P на macOS разыменовывает симлинк во вторую копию молча — копировать через cp -a / git clone / git archive

Гейт — не единственная дешёвая база: ещё три практики заводятся в первый день и не ждут сигнала

1

создай — прожарь — улучши — покажи и согласуй: черновик плана, фиксированный список вопросов (самая простая версия? допущения? преждевременная абстракция? владелец файла? не склеены ли риски?), правки, согласование до начала работы. Живой пример — команда курса: цикл записан в её файле инструкций как обязательный

2

журнал хода работы по задаче, в любом виде — файл на задачу, список шагов, простой лог. Форма не важна, важно, что он есть — всё, что живёт только в сессии, к следующей сессии недоступно (подробный разбор — раздел про память)

3

правило записывается вместе с причиной — строка «почему» рядом с каждым правилом: какой случай его породил. Без причины правило читается как вкусовщина и снимается первым; с названной ценой — это готовый критерий, когда текст пора подпирать механизмом

Ни одна из трёх не требует сигнала и не стоит почти ничего в первый день. Поэтому их и заводят сразу — вместе с гейтом, а не вместо него.

Кейс 1.3 · вложенные файлы

Месяц спустя репозиторий подрос — дробить файл по подпапкам или нет

ФОРМА ФАЙЛА, А НЕ СОДЕРЖИМОЕ



КЕЙС 1.3 · ДВА КУСКА ЗНАНИЯ

За месяц накопились два куска знания разной природы — конвенция целой папки и деталь про одну функцию, — и разработчик поступает с обоими одинаково: заводит вложенные файлы инструкций

Конвенция папки `tests/` — как называются файлы проверок и что со страницей работают через объект страницы, а не через селекторы напрямую. Нужна всякий раз, когда трогают тесты.

Деталь про одну функцию — `submitForm()` в `src/main.js` при таймауте повторяет запрос один раз, поэтому обработчик на сервере обязан быть идемпотентным. Нужна тому, кто правит эту функцию, и больше никому.

что сделал разработчик

`tests/CLAUDE.md` ← конвенция папки

`src/CLAUDE.md` ← деталь про функцию

«Вынесу каждое туда, где оно применяется — короче, релевантнее контексту, корневой файл не разбухнет.»

«Оба файла лежат в подпапках. Расчёт — что агент прочитает их сам, когда будет работать в этих папках.»

КЕЙС 1.3 · ВОПРОС

Куда положить каждый из двух кусков знания, чтобы агент получил его тогда, когда оно нужно, и не платил за него в остальных сессиях

Два куска знания: конвенция для всей папки tests/ и деталь про одну функцию в src/main.js. Карточек можно выбрать несколько.

«Выберите из карточек, куда положить каждый кусок, чтобы агент получил его тогда, когда оно нужно, и не платил за него в остальных сессиях.»

вложенный файл
инструкций в самой
папке

правило с
объявленным
шаблоном путей

комментарий
рядом с самим кодом

отдельный документ +
указатель в корневом
файле

дописать в
корневой файл
инструкций

разбор — на следующем слайде

КЕЙС 1.3 · ИССЛЕДОВАНИЕ (1/2)

«Куда положить» — это вопрос «дойдёт ли текст до агента и сколько он стоит»; у пяти каналов пять разных ответов, и три инструмента из четырёх независимо сошлись на объявленном шаблоне путей

Канал	Когда доходит	Гарантия	Цена контекста
Корневой файл + @-импорты	при старте сессии	есть	платится всегда
Правило с шаблоном путей	когда файл подходит под шаблон	условная — триггер явный	ноль до срабатывания
Комментарий рядом с кодом	когда агент открыл файл	следует из механизма	ноль сверх читаемого
Вложенный файл в подпапке	если инструмент так умеет	НЕТ ни в одном из 4 инструментов	ноль до срабатывания
Отдельный документ без указателя	если агент догадается	нет	ноль — и ноль доставки

Инструмент	Объявленный шаблон путей	«Просто положи файл в папку»
Claude Code	.claude/rules/*.md, поле paths:	вложенный CLAUDE.md — #2571 закрыт «not planned»
Cursor	.cursor/rules/*.mdc, поле globs:	вложенные AGENTS.md — признанный баг
VS Code (Copilot)	.instructions.md, поле applyTo:	вложенные AGENTS.md — экспериментально, выключено по умолчанию
Copilot CLI	—	рекурсивного обнаружения нет вовсе — #3051

Обещание «агенты автоматически читают ближайший файл в дереве каталогов» даёт стандарт agents.md — описание типичного поведения, не требование к инструменту. Обещание не выполняется, потому что его никто и не давал.

КЕЙС 1.3 · ИССЛЕДОВАНИЕ (2/2) И РАЗБОР

Ни одно из двух контролируемых измерений не нашло, что распределение документации улучшает результат — канал выбирается по доставке и цене

Источник	Тезис	Сила
McMillan, 2605.10039 (1650 сессий)	Ни одна из 4 структурных переменных не дала эффекта	Сильное и контринтуитивное
Khatri, 2607.27250 (288 прогонов)	Файл нет/каждый ход/раздельные документы — все в пределах 2 п.п., значимости нет	Сильное по замыслу, слабое по мощности
Vasilopoulos, 2602.20478 (283 сессии)	Единственный построенный целиком свод: ядро + документы по требованию	Слабое — один человек, без контроля
Он же, про режим отказа	«Агенты доверяют документации абсолютно» — устаревание главный режим отказа	Прямое наблюдение владельца

Честно про пробел: «не нашли эффекта» ≠ «эффекта нет»; гарантия комментария в коде выведена из механизма, не измерена; шаблон путей — тоже условная загрузка, не гарантия.

Вариант	Где работает	Почему рано — что вместо
«Вложенный файл в папке»	Нигде гарантированно	Единственный вариант без гарантии во всех 4 инструментах
«Правило с шаблоном путей»	Когда у класса файлов есть конвенция	Верно, но не у нас — конвенция tests/ помещается в 2 строки
«Комментарий рядом с кодом»	Всегда — для знания про код	Часть целевого ответа — единственный канал, где доставка следует из механизма
«Документ + указатель в корне»	Когда документ есть и его поддерживают	Часть целевого ответа — и ответ на «где живёт свод»
«Дописать в корневой файл»	Когда знание нужно в каждой сессии	Presence paradox: строка про tests/ стоит контекста и без тестов

Знание про функцию — в код. Про класс файлов — в правило с шаблоном путей. Про весь проект — в корень, коротко, указателями. Вложенный файл в подпапке — единственное место, куда знание класть не надо.

КЕЙС 1.3 · РЕШЕНИЕ

Корневой файл остался тем же, на тринадцать строк — деталь ушла комментарием к функции, конвенция тестов уже в самих тестах, ни одного вложенного файла инструкций не завелось

```
состояние репозитория, месяц 1

README.md · spec.md · DECISIONS.md
CLAUDE.md      ← те же 13 строк, не выросли
AGENTS.md      → симлинк на CLAUDE.md
index.html · package.json
src/main.js ← комментарий к submitForm(): повтор при
              таймауте, обработчик обязан быть идемпотентным
tests/  form.spec.ts, delivery.spec.ts, page-objects/form.page.ts
# tests/CLAUDE.md и src/CLAUDE.md — не появляются
```

Адрес	Когда включать	Когда рано
Комментарий в коде	сразу, без сигнала	никогда не рано
Правило с шаблоном путей	конвенция класса файлов не помещается в корень	пока конвенция в две строки
Указатель-свод в корне	документ уже есть и его есть кому поддерживать	указатель без документа
Вложенный файл в подпапке	никогда как единственная гарантия	всегда, пока /context не проверен

«Три развилки, один и тот же файл: решили, чего в него не писать заранее; записали гейт готовности, заведённый на дне 0; и решили, что у знания есть адрес — и вложенная папка им не является. Но есть знание другого рода: то, что появляется по ходу самой работы. Это раздел второй — память.»

2

Раздел 2 · память

Кейс 2.1 · плоский файл

Между сессиями агент не хранит ни бита — если это не лежит на диске

- Кейс 2.1 · плоский файл
- Кейс 2.2 · структурированная вики-память
- Кейс 2.3 · операционная память

ЖУРНАЛ РЕШЕНИЙ ПРОЕКТА



КЕЙС 2.1 · ПЛОСКИЙ ФАЙЛ

Конец первой недели: отклонённое решение по валидации всплывает снова — DECISIONS.md заведён на дне 0, но записи об этом решении в нём нет

Конец первой недели работы над signup-landing. Заказчик прислал заявку с опечаткой в домене почты, форма её приняла. Разработчик просит агента ужесточить проверку. Агент предлагает подключить стороннюю библиотеку валидации форм — ту самую, которую обсуждали и отклонили несколько дней назад: форма из двух полей, хватает нативных required/pattern.

«DECISIONS.md заведён на дне 0, в тот же день, что spec.md и README. Записи про библиотеку валидации в нём нет: решение прозвучало в рабочем обсуждении и на диск не попало. Модель между вызовами не хранит ни бита — то, чего нет в файле, во второй сессии не существует.»

Выберите, куда записать отклонённое решение — из карточек ниже — так, чтобы не объяснять его в третий раз

напоминание: отклонённая библиотека валидации всплыла снова — решение родилось в разговоре и нигде не осталось, даже при уже заведённом DECISIONS.md

«Вы второй раз за две недели объясняете агенту, почему в этой форме нет сторонней библиотеки валидации. Выберите, куда записать это решение — из карточек ниже — так, чтобы не объяснять его в третий раз, и обоснуйте, почему не в тот файл инструкций, который мы только что завели.»

дописать
в CLAUDE.md

сказать агенту
«запомни это»

записать в
DECISIONS.md — он уже
есть

поставить систему
памяти

ничего — сам
запомнит

разбор — на следующем слайде

КЕЙС 2.1 · ИССЛЕДОВАНИЕ

Presence paradox из кейса 1.1 применяется и здесь: то, что лежит в DECISIONS.md, стоит контекста в каждой сессии независимо от того, нужна ли эта запись сегодня

Кейс 1.1: Gloaguen et al. — сам факт присутствия текста в контексте создаёт стоимость (+20–23%), независимо от того, помогает ли он решить задачу. Мы назвали это presence paradox.

DECISIONS.md — не исключение. Каждая строка грузится в контекст в каждой следующей сессии, не только когда она реально нужна. Тридцать записей о разных темах — цена уже заметна, даже если задача касается только одной из них.

прямого измерения именно на журналах решений не проводилось — это перенос уже установленного механизма на новый объект, не отдельное новое исследование

если запись не бесплатна — точно ли туда идёт вообще всё, что можно туда положить?

КЕЙС 2.1 · ЧЕТЫРЕ СЛОЯ

Разница между слоями не в формате, а в том, кто инициирует запись, кто её потом увидит и по какому механизму она вообще попадает в контекст

	Файл инстр.	Авто-память	DECISIONS.md	Сложная система
Кто автор	вы, заранее	агент, по ходу	вы, в момент решения	сервис, сам
Где живёт	в репо, git	вне репо, на машине	в репо, git	вне репо, отдельно
Что грузится	целиком, каждую сессию	индекс: 200 строк / 25 КБ	ничего автоматически	ничего автоматически
Код-ревью	да	нет	да	нет
Смена машины	да	нет	да	да, если доступен
Что туда идёт	стабильные правила	предпочтения, подходы	решения и «почему»	что сервис счёл достойным

индекс памяти — только индекс · агент сам пропускает выводимое из кода · сжатие контекста переживают только файлы на диске · усложнение слоя само по себе результата не улучшает

Командное решение с обоснованием идёт в DECISIONS.md, личное предпочтение — в авто-память, выводимое из кода — никуда

вопрос для контекста: «Куда записать отклонённое решение — и почему не в файл инструкций?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Допишу в CLAUDE.md»	Механически сработает: файл читается каждую сессию	Смешивает стабильные правила и растущий лог решений. На десятке решений файл выйдет за ориентир в 200 строк
«Ничего, агент сам запомнит»	Нигде: это не настройка, которую забыли включить	Модель не имеет состояния между вызовами по конструкции
«Скажу агенту ‘запомни это’»	Для личного предпочтения — идеально	Половина ответа. Уходит в авто-память: машинно-локальный слой без синхронизации и код-ревью
«Запишу в DECISIONS.md — он уже есть»	Для командного решения с обоснованием — именно то, что нужно	Вторая половина ответа. Дело не в создании, а в дисциплине: писать «почему», а не «что»
«Поставлю систему памяти»	На десятках-сотнях записей, нескольких пользователях	Рано на порядок: на малых корпусах простой файл регулярно конкурентен со сложными системами

целевой ответ — оба механизма разом, с разделением ролей

Не всё, что узнал агент, вообще стоит записывать — третья корзина такая же законная, как первые две

АВТО-ПАМЯТЬ

Разработчик просит отвечать
коротко, без преамбул

DECISIONS.MD

Не подключаем стороннюю библиотеку
валидации — форма из двух полей

Своего бэкенда не делаем — заявка
уходит на внешний сервис приёма форм

НИКУДА

Обработчик формы лежит
в src/main.js

24 сентября тест упал из-за
таймаута, увеличили ожидание

собранная запись — реальный файл демо-репозитория

2026-09-20 — не подключаем библиотеку валидации форм

Форма — два поля (имя, email), хватает нативных required/pattern в
index.html. Сторонняя библиотека — лишняя зависимость ради этого объёма.

Новое — не файл (он заведён на дне 0), а первая содержательная запись в нём, датированная днём, когда решение записали

DECISIONS.md — целиком

Decisions

Append-only log of why, in date order.

2026-09-26 — сторонние виджеты в форму не добавляем

Форма — два поля (имя, email), хватает нативных required/pattern в index.html. Сторонний виджет — лишняя зависимость и лишние килобайты в сборке ради этого объёма.

состояние репозитория, неделя 1

README.md

spec.md

DECISIONS.md

CLAUDE.md

AGENTS.md

index.html

package.json

src/main.js

tests/form.spec.ts

Память по умолчанию читается как доверенный контекст, а не как данные — и включение памяти не обязательно улучшает результат



Отравление памяти — SpAIware (сентябрь 2024)

Внешний текст с вложенной инструкцией → долговременная память → активация в последующих сессиях, эксфильтрация переписки. Вектор закрыт в версии 1.2024.247; структурный принцип остался.

≈48% всех последующих обращений к памяти захвачено (MemoryGraft, 2025; до отравления — 0%). В смежном эксперименте 5 подложенных документов управляют выводом в >90% случаев.

Замер	Значение	База сравнения
Популярная система памяти на бенчмарке долговременной памяти	46,0% с памятью против 57,6% без	та же система, память выключена
Шесть систем памяти против «весь текст сессии в один файл»	проигрыш плоскому файлу на 25–67 пунктов	тот же плоский файл
Дискреционность записи	17% (4 задачи из 24) — модель отказалась записать нужный факт	те же 24 задачи

Плоского файла достаточно, пока история решений влезает в контекст целиком и связей между записями ещё нет

- Задача одноразовая, второй сессии по этому контексту не будет.
- Весь нужный контекст помещается в одно контекстное окно без потерь.
- Правила проекта стабильны и укладываются в файл инструкций короче 200 строк.
- Никто не повторяет вручную одну и ту же правку из сессии в сессию.
- Нет нескольких участников, которым нужно делиться контекстом асинхронно.
- Факт выводим из кода или истории изменений.

Плоский DECISIONS.md перестаёт справляться, когда записей набралось настолько много, что найти нужную линейным чтением дольше, чем переобъяснить решение заново, — и между записями появились связи, для которых в append-only логе физически некуда положить.

Кейс 2.2 · структурированная вики-память

Тот же DECISIONS.md, четыре месяца спустя

СТРУКТУРА ЖУРНАЛА РЕШЕНИЙ



Четвёртый месяц: в DECISIONS.md двести записей, две из них относятся к одному вопросу — и ни одна не ссылается на другую

Четвёртый месяц работы над signup-landing. На форму пошёл спам; в команде к этому времени работает ещё один разработчик, который прежних обсуждений не застал, и он просит агента поставить защиту.

Агент читает DECISIONS.md — двести записей — и отвечает по записи от 26 сентября: «сторонние виджеты в форму не добавляем». Значит, капча отпадает, и агент предлагает обсуждать вопрос заново.

Запись от 28 ноября — «антиспам делаем honeypot-полем, без капчи» — лежит на полторы сотни записей ниже, ближе к концу файла, и закрывает ровно этот случай. Ни сентябрьская запись не ссылается на ноябрьскую, ни ноябрьская на сентябрьскую.

«Обе записи лежат в одном файле. В плоском append-only логе найти обе можно только линейным чтением всех двухсот.»

Пример иллюстративный — реального DECISIONS.md на 200 записей не захвачено.

Разросшийся журнал решений и потерянная связь между двумя записями — повод выбрать, как его реорганизовать, а не заводить заново

«DECISIONS.md разросся, и в нём уже есть решение, которое отменяет часть более раннего решения, — без единой ссылки между ними. Выберите, как реорганизовать этот файл — из карточек ниже — так, чтобы агент быстро находил нужное решение и видел связи между решениями, а не листал линейный лог с начала. Обоснуйте выбор.»

оставить как есть —
просто грепать

разбить на отдельные
файлы со ссылками

поставить графовую/
векторную систему
памяти

завести один общий
SUMMARY.md,
переписываемый
целиком

агент сам найдёт
через поиск по
репозиторию

разбор — на следующем слайде

Плоская память деградирует со временем, а структура может окупаться именно для агента там, где не окупается для человека — но «умнее» не значит «лучше»

почти все источники ниже — свежие непроверенные препринты 2026 года; цифры — «заявлено в препринте», а не установленный факт

MENTIER (arXiv:2605.03675): плоская память деградирует на горизонте порядка 72 часов работы — минус 14 п.п. успешности вызовов инструментов (насыщение контекста, temporal decay, semantic drift). Логика «raw sources → wiki → schema»: у человека вики умирает — поддержка дороже ценности; у агента стоимость правки полутора десятков файлов близка к нулю.

Система	LoCoMo (долговременная память)
Файловое хранилище (авто-индекс + поиск по смыслу, не только grep)	74,0%
Mem0 (graph-based)	68,5%

Летта-контрпример: простое хранилище обошло графовую систему памяти — модели тренированы на файловых операциях как на самом привычном инструменте.

Разбить на отдельные файлы (по одному на решение) со сквозными ссылками — целевой ответ, но не бесплатный: структура сама создаёт новую нагрузку

вопрос для контекста: «Как реорганизовать разросшийся журнал решений?»

Вариант	Где это работает	Почему здесь ещё рано — и что вместо
«Оставить как есть, просто грепать»	Пока записей десятки и они укладываются в контекст целиком	Деградация плоской памяти со временем задокументирована (~72 ч, -14 п.п.); наш файл уже перерос эту границу
«Разбить на отдельные файлы (по одному на решение) со сквозными ссылками»	Именно этот случай	Целевой ответ, но не бесплатный — структура сама создаёт новую нагрузку (staleness, дублирование, поддержка)
«Поставлю графовую/векторную систему памяти»	На больших многопользовательских корпусах с семантическим поиском	Контрпример прямо про наш масштаб: файловое хранилище обошло graph-based систему на LoCoMo
«Один общий SUMMARY.md, переписываемый целиком»	Для совсем маленького проекта	Переписывание целиком убивает append-only гарантию
«Агент сам найдёт через поиск по репозиторию»	Для фактов, выводимых из кода	Решения — не факты кода. «Сам найдёт» — то же заблуждение, что «сам запомнит»

целевой ответ — разбить на отдельные файлы (по одному на решение) со сквозными ссылками, со списком реальных издержек, а не как безусловный апгрейд

КЕЙС 2.2 · РЕШЕНИЕ

Практика ADR: одно решение — один нумерованный файл, а связь между решениями — строка в разделе Status, которую инструмент проставляет сразу в обе стороны

```
doc/adr/  
0001-record-architecture-...md  
...  
0031-storonnije-vidzhety-...md  
...  
0187-antispam-honeypot-...md  
README.md ← adr generate toc
```

ноябрьская запись — по шаблону adr-tools

187. Антиспам — honeypot-поле, без капчи

Date: 2026-11-28

Status

Accepted

Amends [31. Сторонние виджеты не добавляем]

Context / Decision / Consequences

...

`adr new -I "31:Amends:Amended by" ...` — и в файл 31 инструмент сам дописал встречную строку `Amended by [187...]`. Обе ссылки написал инструмент, а не автор записи.

Источник практики: ADR — Майкл Найгард, 2011; adr-tools. Сила доказательства: задокументированная практика для людей (4500+ ADR в 750 проектах), но контролируемого измерения «ADR помогают агенту» нет. Содержимое записей иллюстративное, ждёт живой сессии.

КЕЙС 2.2 · КОНТРПРИМЕР: ЧТО ИМЕННО ИЗМЕРИЛИ

Файловое хранилище обошло графовую память — но измеряли вопросы по длинному диалогу, а не поиск связанного решения в журнале проекта

LoCoMo — бенчмарк памяти в очень длинных диалогах (Maharana и др., 2024): переписка двух собеседников, до 35 сессий, ~300 реплик, ~9 тыс. токенов. Доля вопросов о фактах переписки с верным ответом.

Система	LoCoMo (долговременная память)
Файловое хранилище Letta (авто-индекс + поиск по смыслу, не только grep)	74,0%
Mem0, графовый вариант (число из публикации Mem0)	68,5%

объяснение авторов: модели тренированы на файловых операциях как на самом привычном инструменте

ЧЕГО ЭТОТ РЕЗУЛЬТАТ НЕ ГОВОРИТ
Это не один прогон — числа из двух разных публикаций; авторы Letta сами называют такое сравнение «яблоки с апельсинами». · «Файловое хранилище» — не «только grep»: файл индексируется, поиск по смыслу у агента был. · Задача другая: измеряли вопросы о фактах переписки, а не поиск связанного решения в журнале решений проекта. Наш случай никто не мерил.

Четыре издержки структуры — не общие категории, а конкретные файлы нашего же doc/adr/: сгенерированное оглавление, лишняя запись, осиротевшее обоснование и ручная сверка



Устаревание (staleness)

README.md собран командой adr generate toc в октябре. Записи с 0188 по 0200 в него не попали, а выглядит он полным.



Дублирование (duplication)

Кто-то не нашёл 0187 и завёл 0193 «Пороги антиспама» про тот же honeypot. Обе Accepted, ссылки между ними нет.



Нет каскадного удаления

0031 отменили — статус и ссылки перевелись. Но 0187 обоснована фразой «запрещено решением 31» и осталась Accepted.



Нагрузка на поддержку

Двести решений — двести файлов: номер, дата, статус, ссылки, регенерация оглавления. Заметить дубль не делает никто.

У человека эта нагрузка обычно убивает вики — наблюдение, не измерение. У агента правка полутора десятков файлов за проход стоит почти ноль — но это гипотеза о механизме. Структура — компромисс, не бесплатный апгрейд.

КЕЙС 2.2 · КРИТЕРИЙ И МОСТ

Единицы связей на запись и один-два человека, правящих память, — структуры достаточно; полноценная система памяти требует замеров на своём масштабе, не общего совета

- Связей у записи — единицы, а не десятки: их видно в разделе Status.
- Поиск нужной записи — по номеру, названию файла или простому grep, а не по смысловой близости.
- Участников, одновременно правящих память, — один-два.
- Никто не просил «найди решения, похожие по смыслу на...».

Смена оси: всё в этом и предыдущем кейсе — про решения, которые живут навсегда. Что происходит с памятью одной задачи, пока она ещё не закончена?

Кейс 2.3 · операционная память

Смена оси: не решения навсегда, а ход одной задачи между её сессиями

ПАМЯТЬ ОДНОЙ ЗАДАЧИ



КЕЙС 2.3 · ЗАДАЧНАЯ ПАМЯТЬ

Файл задачи заведён на дне 0 и план в нём отмечен по первым шагам — а договорённость о том, как делать следующий шаг, принятая по ходу первой сессии, в него не попала

Практика «файл на задачу» заведена на дне 0, вместе с DECISIONS.md: на каждую многошаговую задачу — свой файл с планом и логом хода. Задача: разбить форму на пошаговый визард — имя, email, подтверждение, — сохранив всю текущую валидацию. Файл создан перед началом работы, план расписан на шесть шагов.

Первая сессия закрывает шаги 1-2. По ходу шага 2 видно, что значения полей надо где-то держать между шагами, и в рабочем обсуждении — не в плане — выбирается способ: состояние визарда хранить одним объектом в sessionStorage, ключ signup-wizard. В лог хода выбранный способ не попадает. Сессия обрывается в середине шага 3 — контекст сжимается.

Вторая сессия. Агент открывает файл: план на месте, шаги 1-2 отмечены выполненными. Про sessionStorage в файле нет ни строки. Агент делает шаг 3 самым коротким способом — держит состояние в переменной модуля. Тесты шага 5 зелёные. Расхождение всплывает на шаге 6: перезагрузка страницы на третьем шаге очищает форму.

«На входе во вторую сессию у агента есть ровно то, что лежит в файле: шесть шагов плана и две отметки о выполнении. Договорённость про хранение состояния осталась в контексте первой сессии, которого больше нет.»

Файл задачи существовал с самого начала и всё равно ввёл в заблуждение — выбор из пяти вариантов

«Файл задачи существовал с самого начала, план в нём выглядит актуальным — первые шаги отмечены сделанными. И всё равно вторая сессия делает шаг 3 не так, как договорились в первой. Выберите, что нужно сделать иначе — из карточек ниже — так, чтобы файл задачи не вводил в заблуждение, а реально отражал ход работы. Обоснуйте выбор.»

надеяться на
контекстное окно

записать план
в DECISIONS.md

правило в CLAUDE.md:
договорённость — сразу
в файл

завести папку на задачу
с файлом под каждый
шаг

попросить агента
«продолжи с того
места»

разбор — на следующем слайде

КЕЙС 2.3 · ИССЛЕДОВАНИЕ

От Pokémon до recitation — практика структурированных заметок о ходе задачи известна и работает; и она объясняет, почему наш файл всё равно подвёл

Anthropic, инженерный блог: «structured note-taking» — агент играет в Pokémon тысячи шагов подряд, ведя простые структурированные заметки, переживая сбросы контекста без потери прогресса. Manus, «recitation»: агент непрерывно переписывает todo.md в конец контекста — снижает уход от цели. metasphere-agents: файл на задачу в .tasks/active/, архивируется в .tasks/done/ по завершении.

«Есть измерение на большом числе траекторий: план в среднем помогает. Но дословная цитата авторов — ‘плохой план вреднее отсутствия плана вообще’. Полную картину с цифрами покажу на слайде провала.»

Целевой ответ — строка в CLAUDE.md, обязывающая агента дописывать договорённость в файл задачи в момент, когда она прозвучала

вопрос для контекста: «Файл задачи ввёл в заблуждение — что сделать иначе?»

Вариант	Где это работает	Почему рано — и что вместо
«Не сжимать, не закрывать сессию»	Для задачи, целиком укладываемой в одну короткую сессию	Контекстное окно конечно; на многошаговой задаче рано или поздно случится сжатие или разрыв сессии — это отсутствие стратегии
«Записать план в DECISIONS.md»	Никогда — не тот слой памяти	DECISIONS.md — про решения, которые остаются в силе навсегда; статус одной задачи туда не относится
«Правило в CLAUDE.md: договорённость — сразу в файл задачи»	Ровно наш случай — файл уже есть, не хватило строки в инструкциях	Целевой ответ. Адресат — агент, который файл и так ведёт, не память разработчика
«Папка на задачу с файлом под каждый шаг»	Параллельные подзадачи, несколько субагентов, разросшийся лог	Оверинжиниринг для одной линейной задачи здесь и сейчас
«Попросить ‘продолжи с того места’»	Внутри одной непрерывной сессии, до сжатия	После сжатия или в новой сессии буквально нечего продолжать

целевой ответ — строка в CLAUDE.md, адресованная агенту, с явной оговоркой об упрощении до файла

Новое в этом кейсе — не файл, а три строки в CLAUDE.md: они говорят агенту, когда в этот файл писать

CLAUDE.md — добавлено

Задачная память

- файл .tasks/active/<задача>.md:
план, лог хода, результаты
- договорённость дописывать в
«Лог хода» сразу, тем же ответом
- в начале и в конце сессии сверять
план с репозиторием, строка
«Сверено: <дата>»

.tasks/active/signup-wizard.md

Задача: разбить форму на пошаговый визард

План

1. [x] Разметка трёх шагов
2. [x] Перенос текущей валидации
3. [] Состояние текущего шага (JS)

Лог хода

- 2027-01-26: шаги 1-2 закрыты. Состояние визарда — sessionStorage, ключ signup-wizard. Решено по ходу, в плане нет.
- 2027-01-28: начат шаг 3 — контекст сжат.

Сверено

2027-01-28: план сверен, расхождений нет.

иллюстративный пример, не захваченный вживую · правило в тексте — просьба, не закон · упрощение до файла — масштабируется до папки, подробнее за пределами этого занятия

Плохой план хуже отсутствия плана вообще — и это подтверждено количественно, а не только интуицией

Источник	Результат
Evaluating Plan Compliance, 21 120 траекторий, SWE-bench Verified/Pro	«bad plans hurt performance more than no plan at all»; комплаенс к плану падает ~на 13 п.п. на сложном бенчмарке
Fresh Memory, Stale Plans, 30 контролируемых живых сценариев	без проверки актуальности плана исполнитель действовал по устаревшему плану во всех 30 случаях (100%)



Кейс-стади «Index Sickness» (n=1, честная оговорка)

Реальный проект, 391 сессия за месяц. Решение — физическое разделение baseline-инструкций и append-only лога сессии. Результат: объём AI-инструкций сократился ~на 75%, рецидивов не было за следующие ~150 сессий. n=1 action research одного автора, не контролируемый эксперимент.

практика «файл/папка на задачу» пока не мейнстрим — 2853 реальных репозитория, доминируют статические context-файлы

КЕЙС 2.3 · КРИТЕРИЙ

Одного файла на задачу достаточно, пока задача линейная и её ведёт один агент и один разработчик — иначе оправдан переход к папке

- Задача линейная: шаги идут по порядку, не параллельно.
- Работает один агент (без субагентов/параллельных веток) и один разработчик.
- Лог хода помещается в разумный файл (сотни строк, не тысячи) без потери читаемости.
- Результатов и артефактов на выходе немного — не нужен отдельный файл на каждый.

Если хотя бы половина пунктов неверна — переход к папке на задачу оправдан.

честная оговорка: паттерн «файл/папка на задачу» пока не мейнстрим (2853 реальных репозитория, доминируют статические context-файлы) — практику продвигают отдельные лидеры, это не общепринятая норма

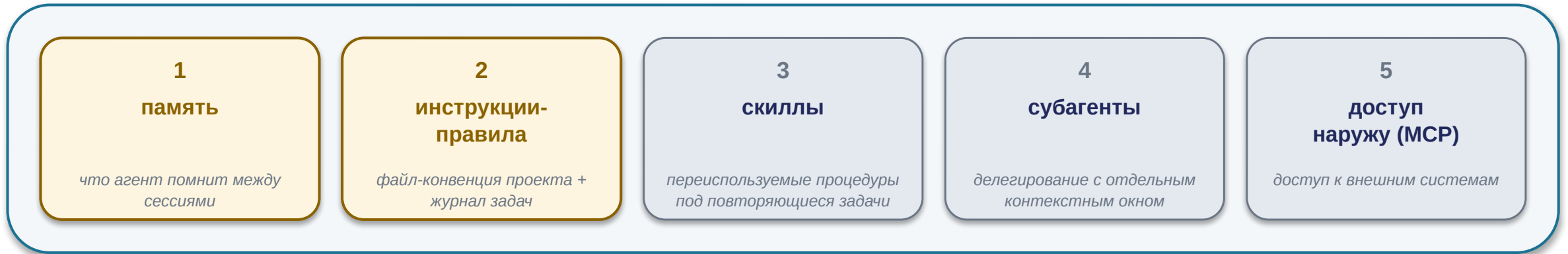
РАЗДЕЛ 3 · СИНТЕЗ

Шесть развилок сегодняшнего занятия делились на два типа: специфичное для проекта (ждать сигнала) и универсальная гигиена (заводить сразу, на день 0)

Кейс	Развилка	Что мы записали	Когда
1.1	Роль агента и общее описание репозитория	Ничего — до первого реального сигнала	по сигналу
1.2	Базовые процессы: гейт «готово» + цикл улучшения	Гейт-фраза текстом в CLAUDE.md	сразу
1.3	Вложенные файлы инструкций по подпапкам	Правило трёх адресов: функция → код, класс файлов → шаблон, проект → корень	по сигналу
2.1	Плоский файл памяти (DECISIONS.md)	Решение записано в файл + авто-память	сразу
2.2	Структурированная вики-память	Отдельные файлы (ADR) со сквозными ссылками	по сигналу
2.3	Операционная задачная память	Один файл на задачу: план + лог + результаты	сразу

Три развилки ждали реального сигнала. Три — не нуждались в сигнале вообще, их стоило завести сразу.

Сегодня закрыты подробно два слота харнесса из пяти; оставшиеся три плюс оба механических слоя, которые превращают просьбу в барьер, — материал Семинара 5



Семинар 5: скиллы · субагенты · доступ наружу (MCP) + хук · процесс

Перенесите шесть сегодняшних решений на свой репозиторий — и проверьте, какое из них реально день-0-база, а какое вы отложили без причины или завели раньше времени без сигнала

1.1 → 1.2 → 1.3 → 2.1 → 2.2 → 2.3

«По каждой развилке спросите: это действительно день-0-база — то, что стоило завести сразу, независимо от конкретики проекта? Или вы завели это раньше времени, без реального сигнала, просто по привычке? А может, наоборот — что-то из универсальной гигиены вы всё ещё откладываете без всякой причины?»

«Не вслух. Не на бумаге. Себе.»